SCS1207
Software Engineering I

H N D THILINI

Design & Implementation

Learning Objectives

- Understand the most important activities relevant to design and implementation.
- Define the term design pattern and the importance of this activity.
- Understand the key concepts such as software reuse and open source development

Topics Covered

- Object-oriented design using the UML
- Design patterns
- Implementation issues
- Open source development

Design and implementation

- Develop an executable software system.
- Software design and implementation activities are invariably inter-leaved.
  - Software design: a creative activity in which you identify software components and their relationships, based on a customer's requirements. (how to solve the problem)
  - Implementation: the process of realizing the design as a program.

1. Object Oriented Design

- Introduction to Object oriented design
- Key OO Concepts
- Common activities involved in OO analysis and design
- Advantages & Challenges in Object Oriented Design

2. Design Patterns

- What is a design Pattern
- Main elements of a design pattern description (explain with 'observer' design pattern as example)
- Creational Design Patterns
- Structural Design Patterns
- Behavioral Design Patterns

3. Implementation Issues

- Reuse
  - Levels of reuse
  - Costs of Reuse
- Configuration Management
- Host-Target Management

4. Open Source Development

- What is Open Source Development
- Open Source Licencing
- License Management

Object-oriented design using UML

Object Oriented Design

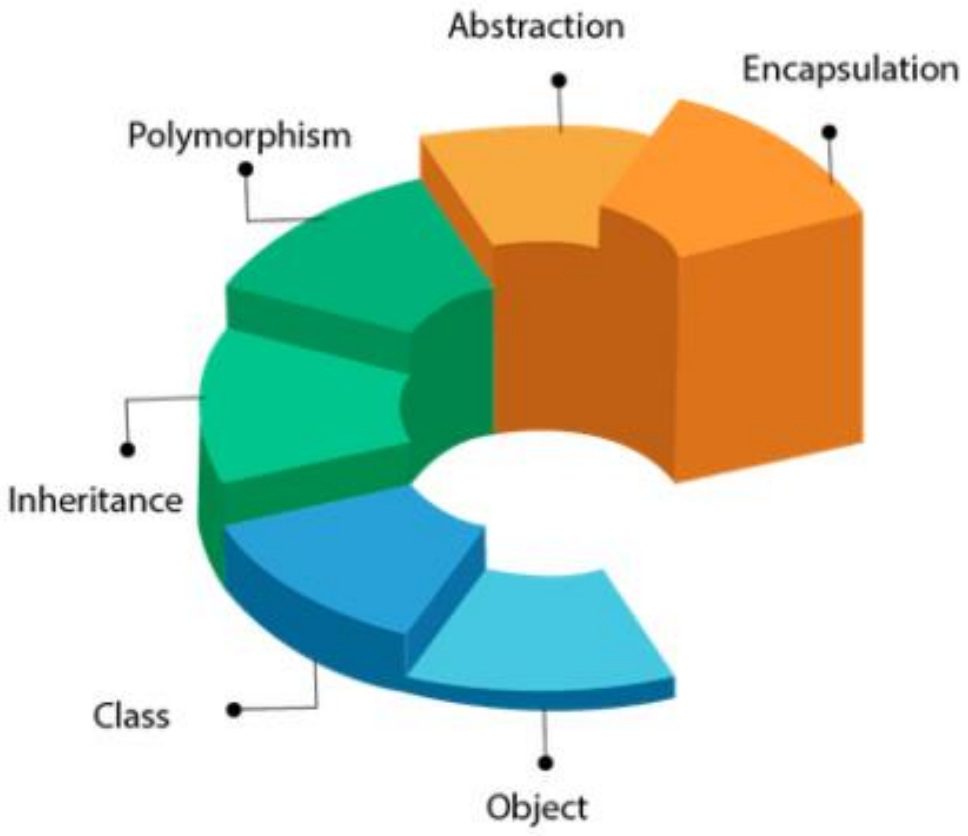
- Promotes the component- based approach to software development.
  - Create a system by creating a set of objects.
  - Expand the system by adding capabilities to components that have already built *or* by adding new components.
  - Can reuse the objects when building new system.
- Substantially reduce the system development time.

An object-oriented design (OOD)

- Designed using several interacting objects.
- Design object classes and the relationship between these classes.
- Objects are created dynamically from the class definitions.
- Objects are stand-alone entities.
- Changing the implementation of an object or adding services should not affect other system objects.
- Advantages of OOD?
  - Improves understandability
  - Increase maintainability

OO Concepts

- Abstraction
- Inheritance
- Polymorphism
- Encapsulation



OO Concepts

- **Abstraction**
  - Abstraction is about hiding unwanted details while showing most essential information.
  - A mechanism to reduce and filter out details so that one can focus on a few concepts at a time.
  - focuses on ***what an object does***, instated of how an object is represented or "*how it works*."
  - Often used for managing large and complex programs.

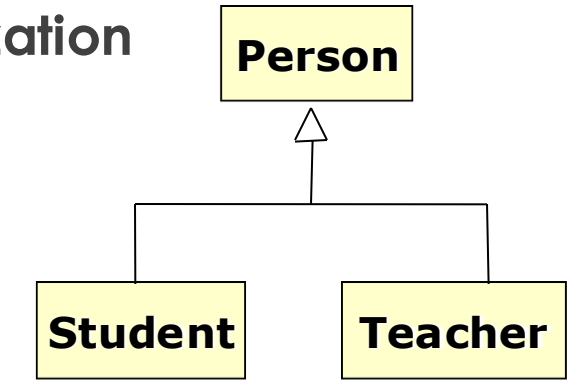
E.g: Human Being's can talk, walk, hear, eat, but the details of the muscles mechanism and their connections to the brain are hidden from the outside world.

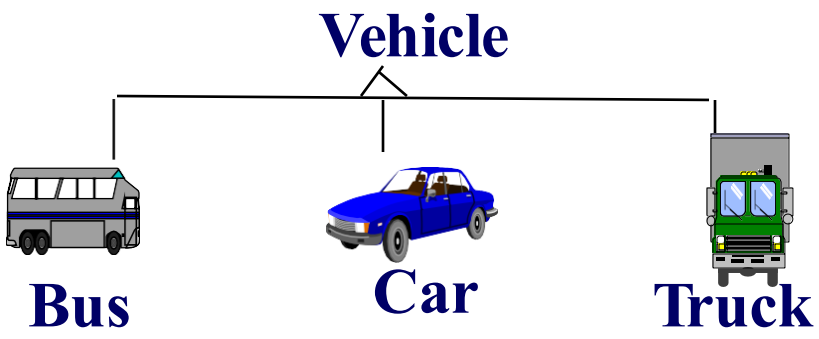
OO Concepts

- **Inheritance**
  - Methods and/or attributes defined in an object class can be inherited or reused by another object class.

E.g: some individuals in the room might be classified as STUDENTS and TEACHERS. Thus, STUDENT and TEACHER object classes are members of the object class PERSON

- **Generalization / Specialization**





* Specialized classes inherits from the parent class

OO Concepts

Generalization

Person

firstName
lastName
birthdate
gender

walk
jump
talk
sleep

Inheritable
Attributes
And
behavior

Specialization

Student

GPA
Classification

enroll
displayGPA

Teacher

rank

lecture

+

firstName
lastName
birthdate
gender
walk
jump
talk
sleep

OO Concepts

- **Polymorphism**
  - Allows different forms of the same service to be defined.
  - Sometimes an operation has the same name in different classes.

E.g: You can open a door, you can open a window, and you can open a newspaper, a bank account or a conversation. In each case, you are performing a different operation.

OO Concepts

- **Encapsulation**
  - Packaging of several items together into one unit (both attributes and behavior of the object)
  - The only way to access or change an object's attribute is through that object's specific behavior.
  - Objects *encapsulates* what they do.
    - That is, they hide the inner workings of their operations
      - from the outside world
      - and from other objects
  - Also known as information hiding

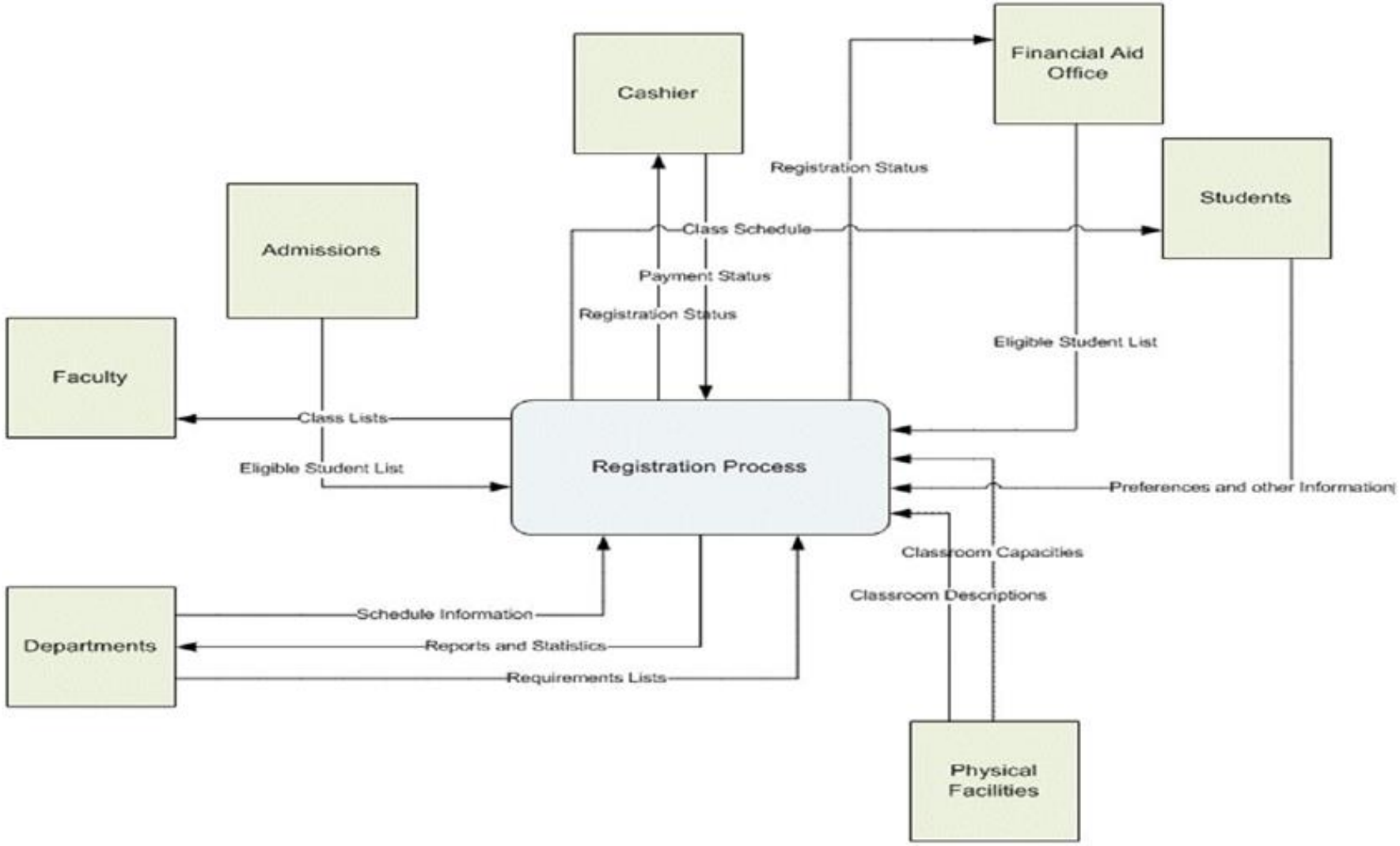
OOD Process stages

- There are a variety of different object-oriented design processes that depend on the organization.
- Common activities in these processes include:
  1. Define the context;
  2. Design the system architecture;
  3. Identify the principal system objects;
  4. Develop design models;
  5. Specify object interfaces.

1. System context and interactions

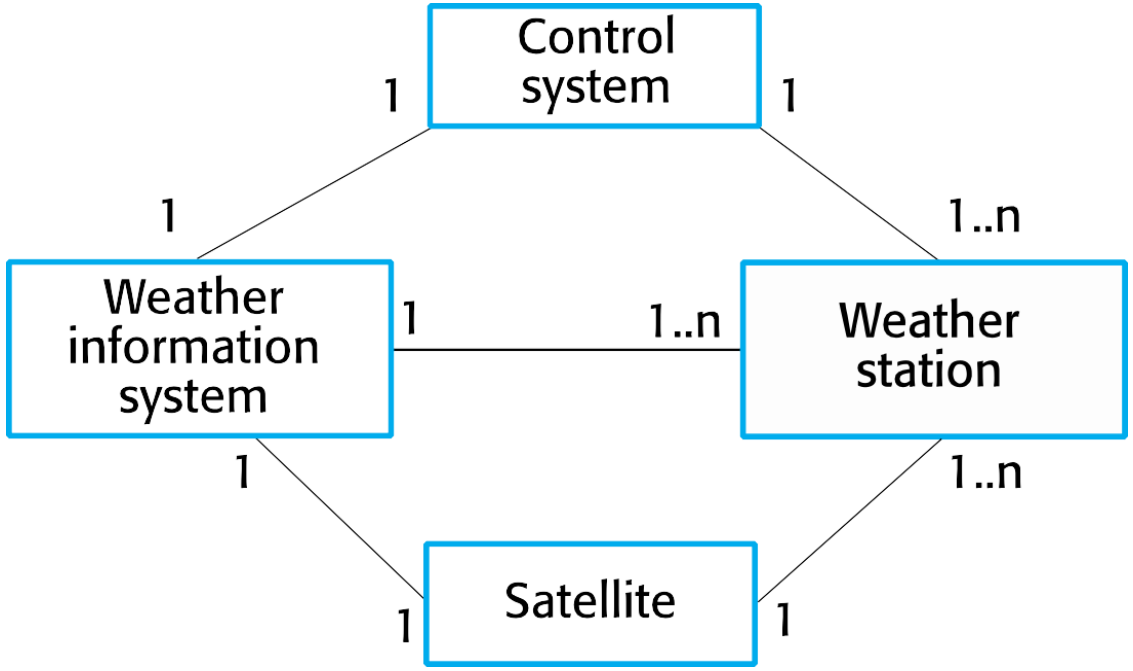
- The relationships between the software that is being designed and its **external environment**
  - essential for deciding how to provide the required system functionality and how to **structure the system to communicate with its environment**.
- **Establish the boundaries of the system**.
  - decide what **features** are implemented in the **system** being designed and what features are in other **associated systems**.
- A system context model demonstrates the **other systems** in the environment of the system being developed and how the system interacts with its environment.

1. System context and interactions



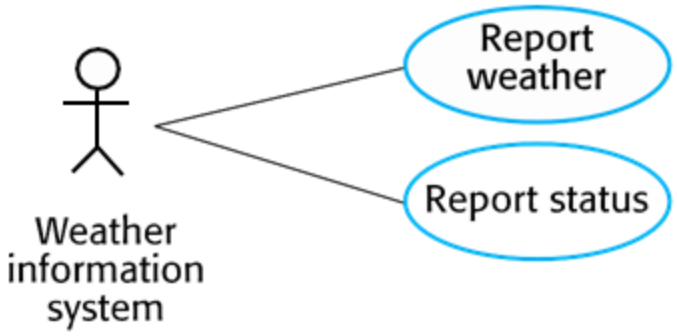
1. System context and interactions

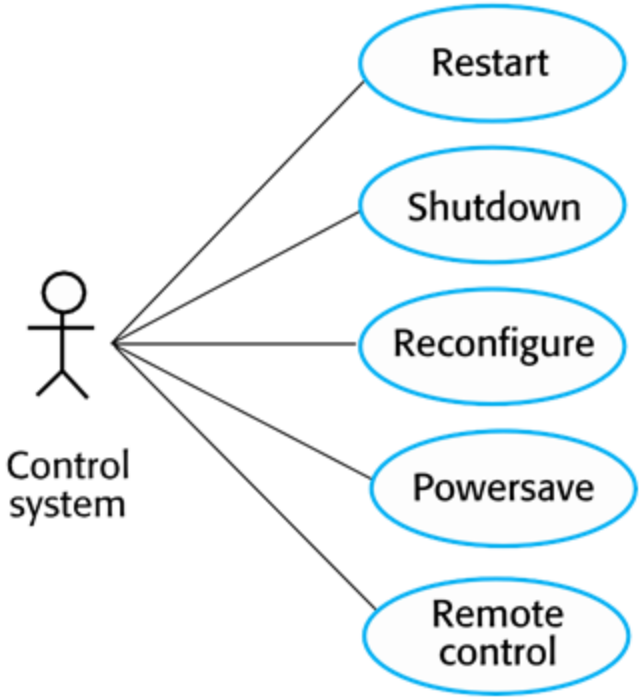
- System context for the weather station



1. System context and interactions

- Weather station use cases



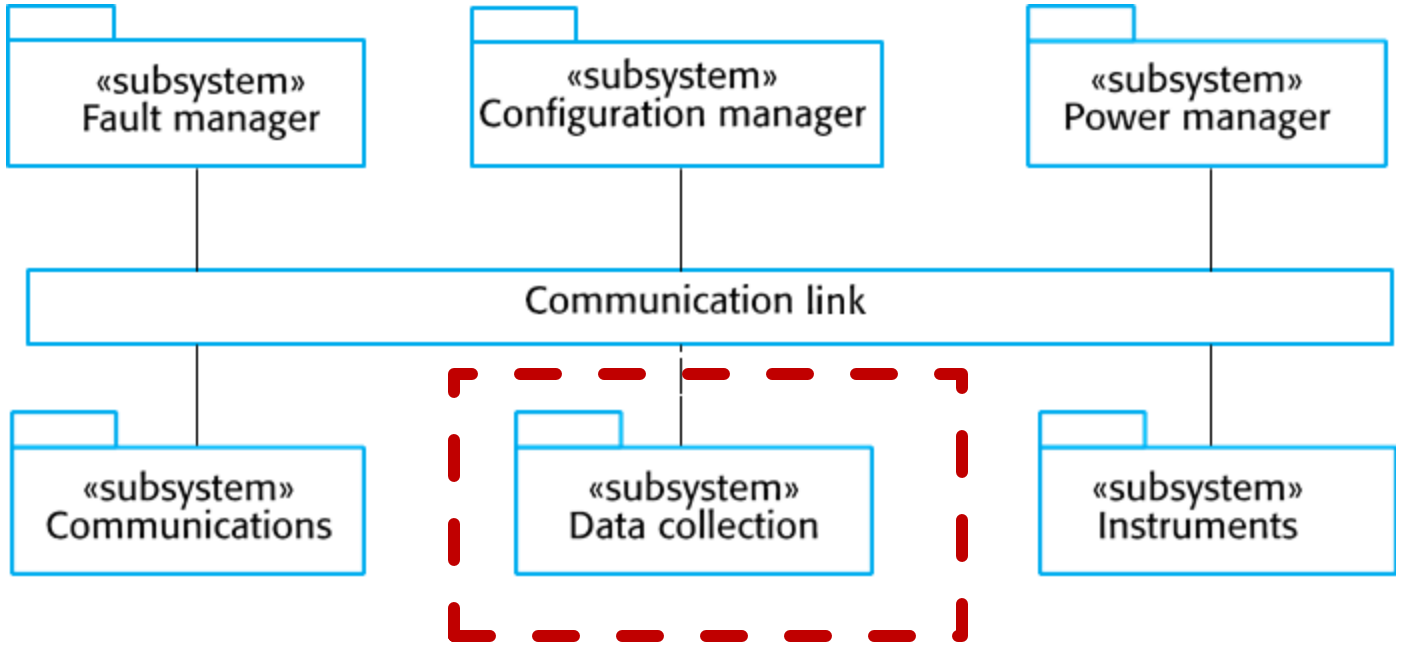


2. Architectural Design

- Identify the major components that make up the system and their interactions.
- Organize the components using an architectural pattern such as a layered or client-server model.
- e.g: The weather station is composed of independent subsystems that communicate by broadcasting messages on a common infrastructure.

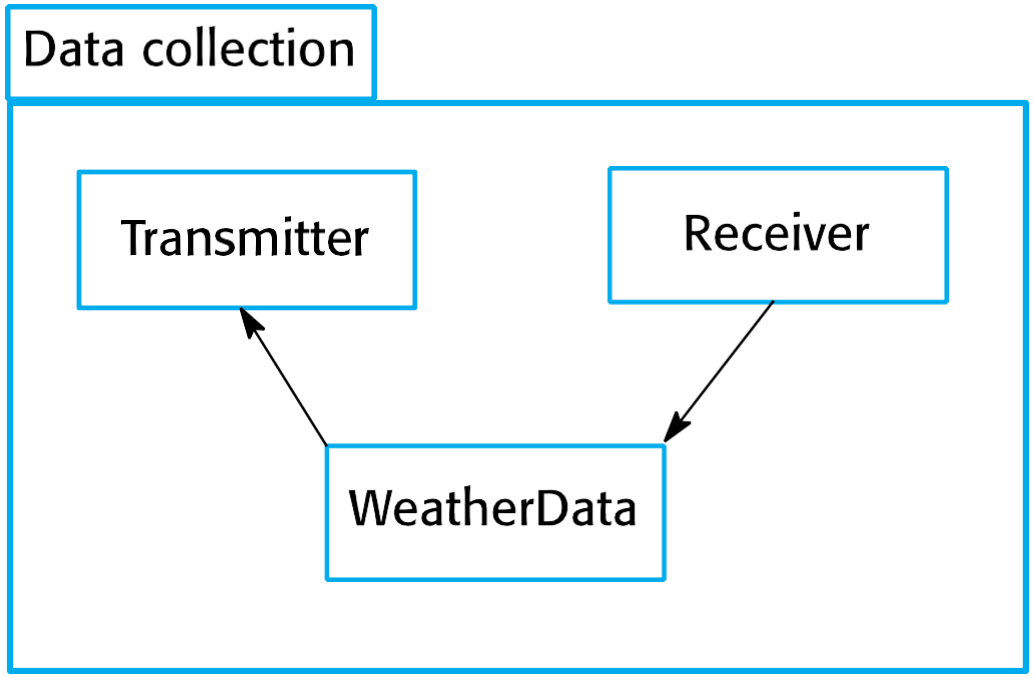
2. Architectural Design

- High-level architecture of the weather station



2. Architectural Design

- Architecture of data collection system

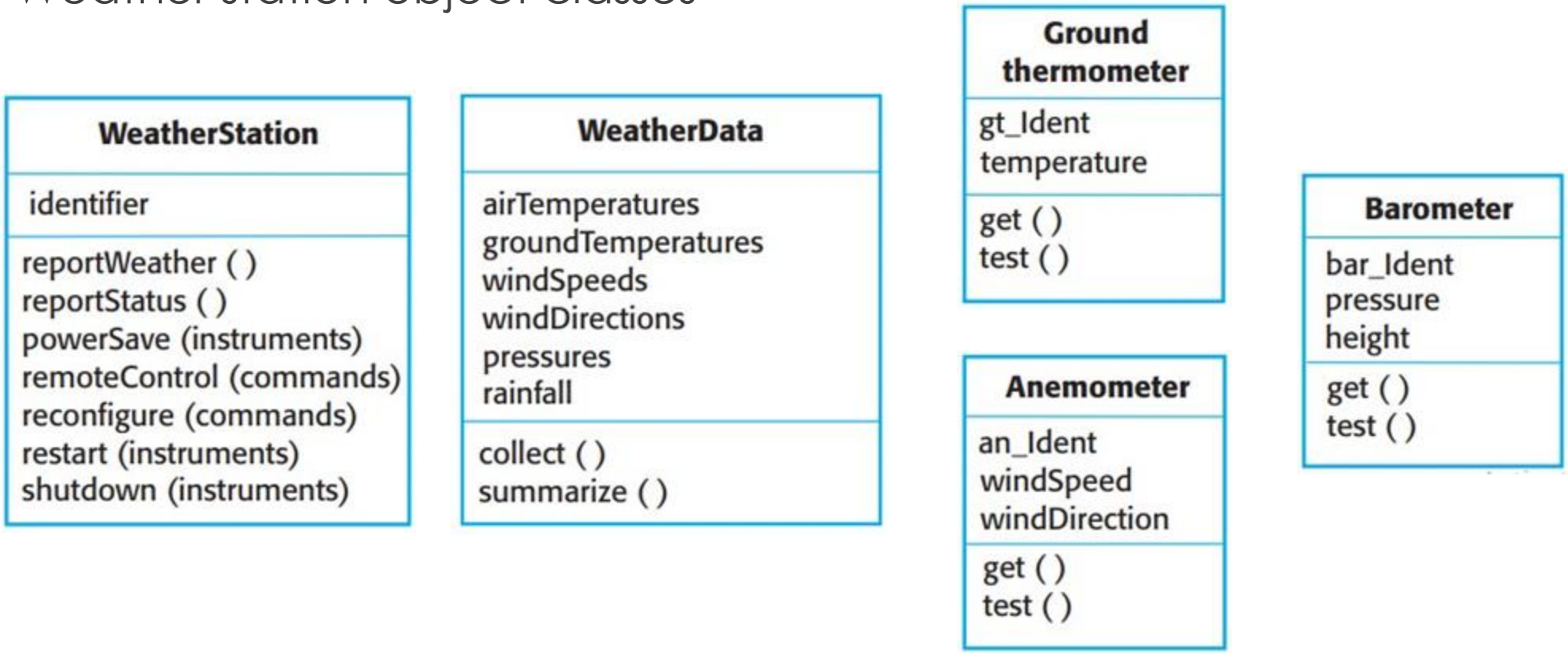


3. Object Class Identification

- Identifying object classes is often a difficult part of object oriented design.
- There is no 'magic formula' for object identification. It relies on the skill, experience and domain knowledge of system designers.
- Object identification is an iterative process. You are unlikely to get it right first time.

3. Object Class Identification

- Weather station object classes



4. Design Models

- Design models show the objects and object classes and relationships between these entities.
- There are two kinds of design model:
  - **Structural models:** describe the static structure of the system in terms of object classes and relationships.
  - **Dynamic models:** describe the dynamic interactions between objects.

4. Design Models

- Examples of design models
  - Subsystem models:
    - show logical groupings of objects into coherent subsystems.
  - Sequence models:
    - show the sequence of object interactions.
  - State machine models:
    - show how individual objects change their state in response to events.
  - use-case models, aggregation models, generalization models, etc.

4. Design Models

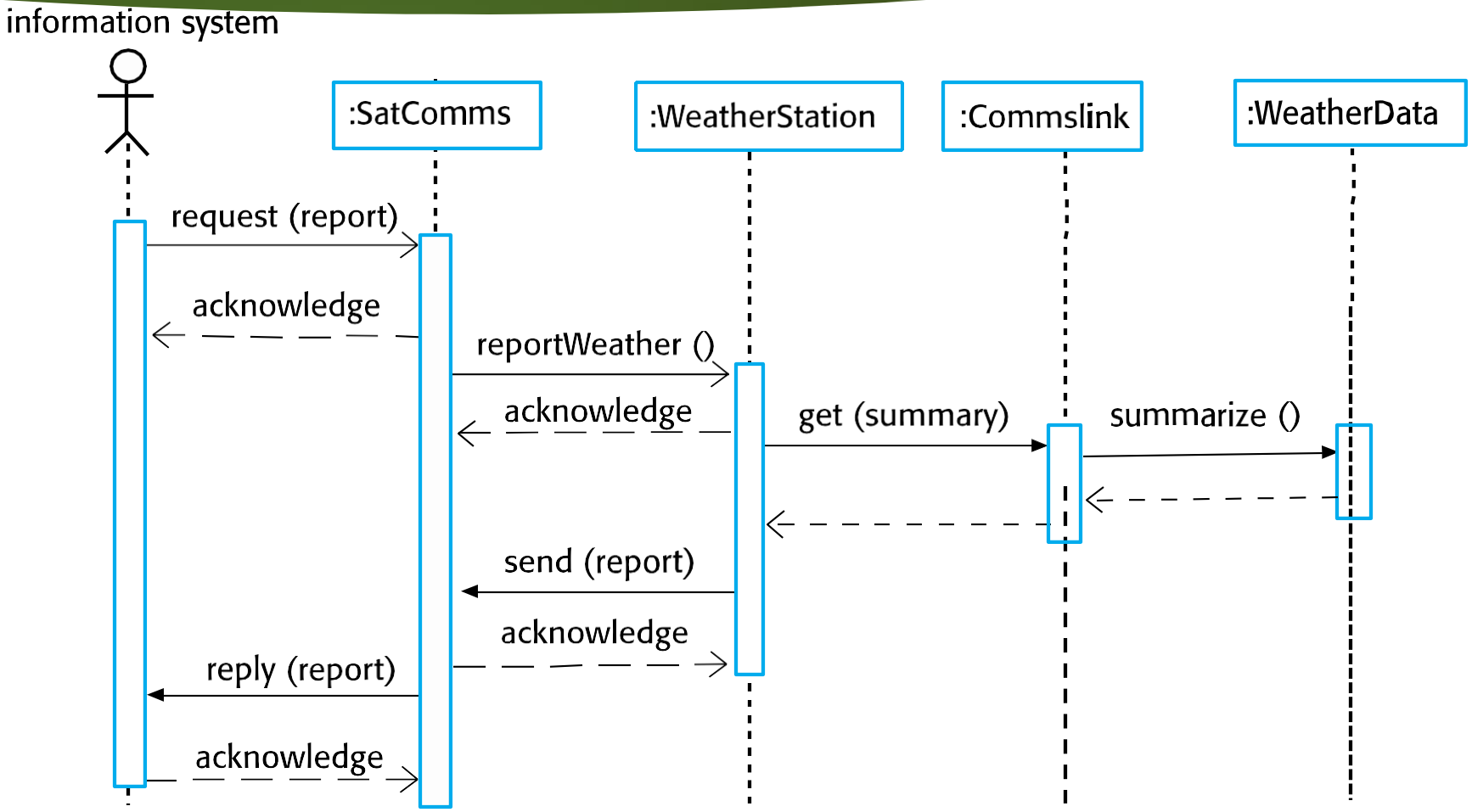
- Subsystem models
  - Shows how the design is organized into logically related groups of objects.
  - In the UML, these are shown using **packages** - an encapsulation construct. This is a logical model. The actual organization of objects in the system may be different.

4. Design Models

- Sequence models
  - Show the sequence of object interactions
    - Objects are arranged horizontally across the top;
    - Time is represented vertically so models are read top to bottom;
    - Interactions are represented by labelled arrows, Different styles of arrow represent different types of interaction;
    - A thin rectangle in an object lifeline represents the time when the object is the controlling object in the system.

4. Design Models

- Sequence diagram describing data collection

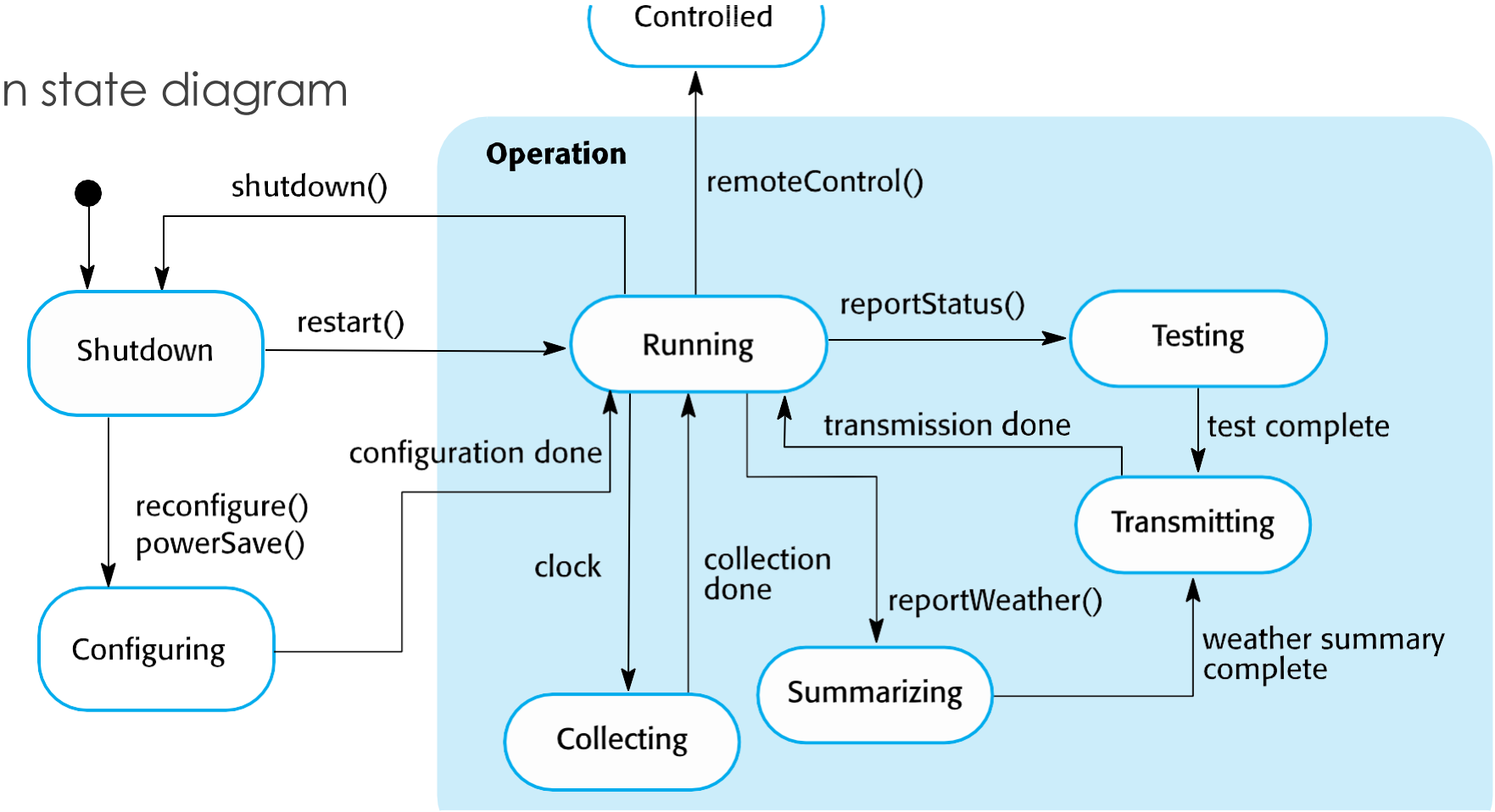


4. Design Models

- State diagrams
  - Show how objects respond to different service requests and the state transitions triggered by these requests.
  - State diagrams are useful high-level models of a system or an object's run-time behavior.
  - State diagrams for all of the objects in the system are NOT required.
  - Many of the objects in a system are relatively simple and a state model adds unnecessary detail to the design.

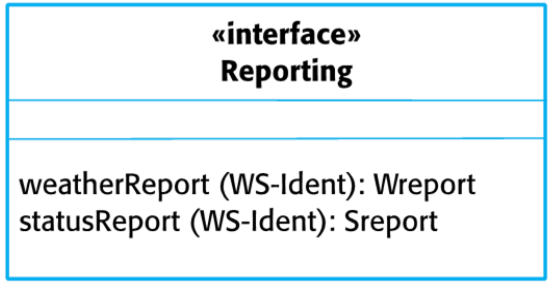
4. Design Models

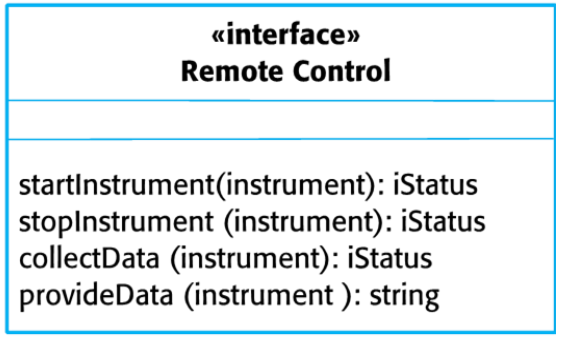
- Weather station state diagram



5. Interface Specification

- Object interfaces have to be specified so that the objects and other components can be designed in parallel.
- Designers should avoid designing the interface representation but should hide this in the object itself.
- Objects may have several interfaces which are viewpoints on the methods provided.





Design patterns

Design Patterns

- A design pattern is a way of reusing abstract knowledge about a problem and its solution.
- A pattern is a description of the problem and the essence of its solution.
- It should be sufficiently abstract to be reused in different settings.
- Pattern descriptions usually make use of object-oriented characteristics such as inheritance and polymorphism.

Patterns

- *Patterns and Pattern Languages are ways to describe best practices, good designs, and capture experience in a way that it is possible for others to reuse this experience.*
- A **design pattern** is a general repeatable solution to a commonly occurring problem in software **design**.
- A **design pattern** isn't a finished **design** that can be transformed directly into code.
- It is a **description** or **template** for **how to solve a problem** that can be **used in many different situations**.

Main elements of a design pattern description

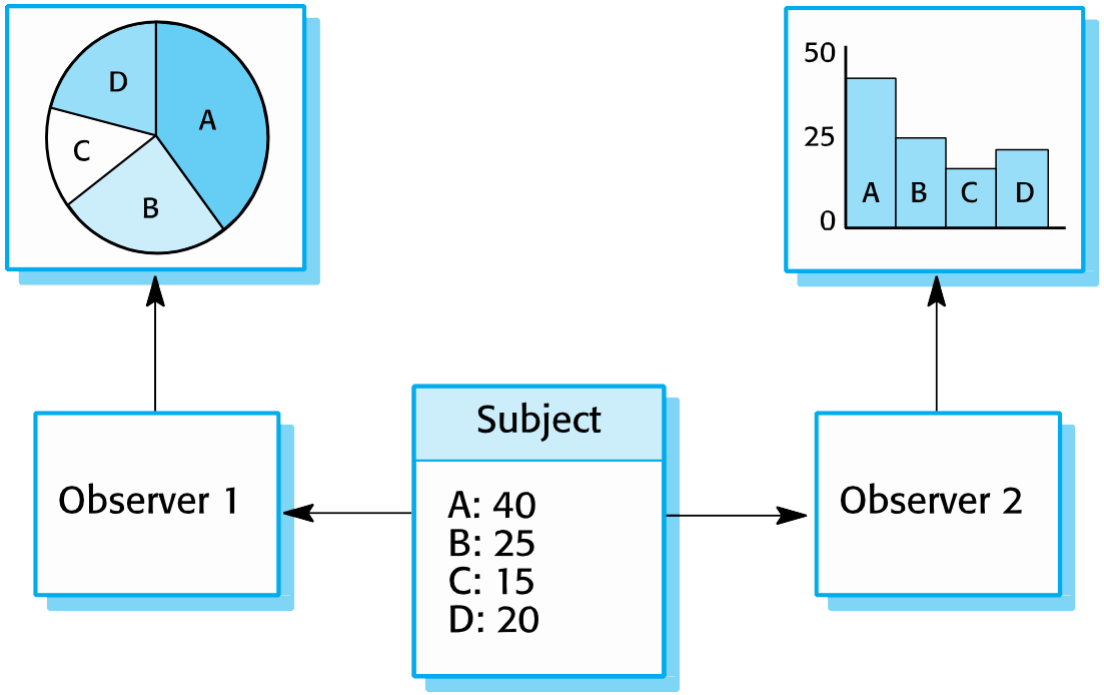
- Name
- Problem Description
- Solution Description
- Consequences

Pattern Elements

1. Name
   - A meaningful pattern identifier.
2. Problem description.
3. Solution description.
   - Not a concrete design but a template for a design solution that can be instantiated in different ways.
4. Consequences
   - The results and trade-offs of applying the pattern.

Design Pattern: Example

1. Name : Observer Design Pattern



Multiple displays using the Observer pattern

Design Pattern: Example

2. Description
   - **Separates the display** of the state of an object from the **object itself** and **allows alternative displays** to be provided. When the **object state changes**, all **displays are automatically notified and updated** to reflect the change.

Design Pattern: Example

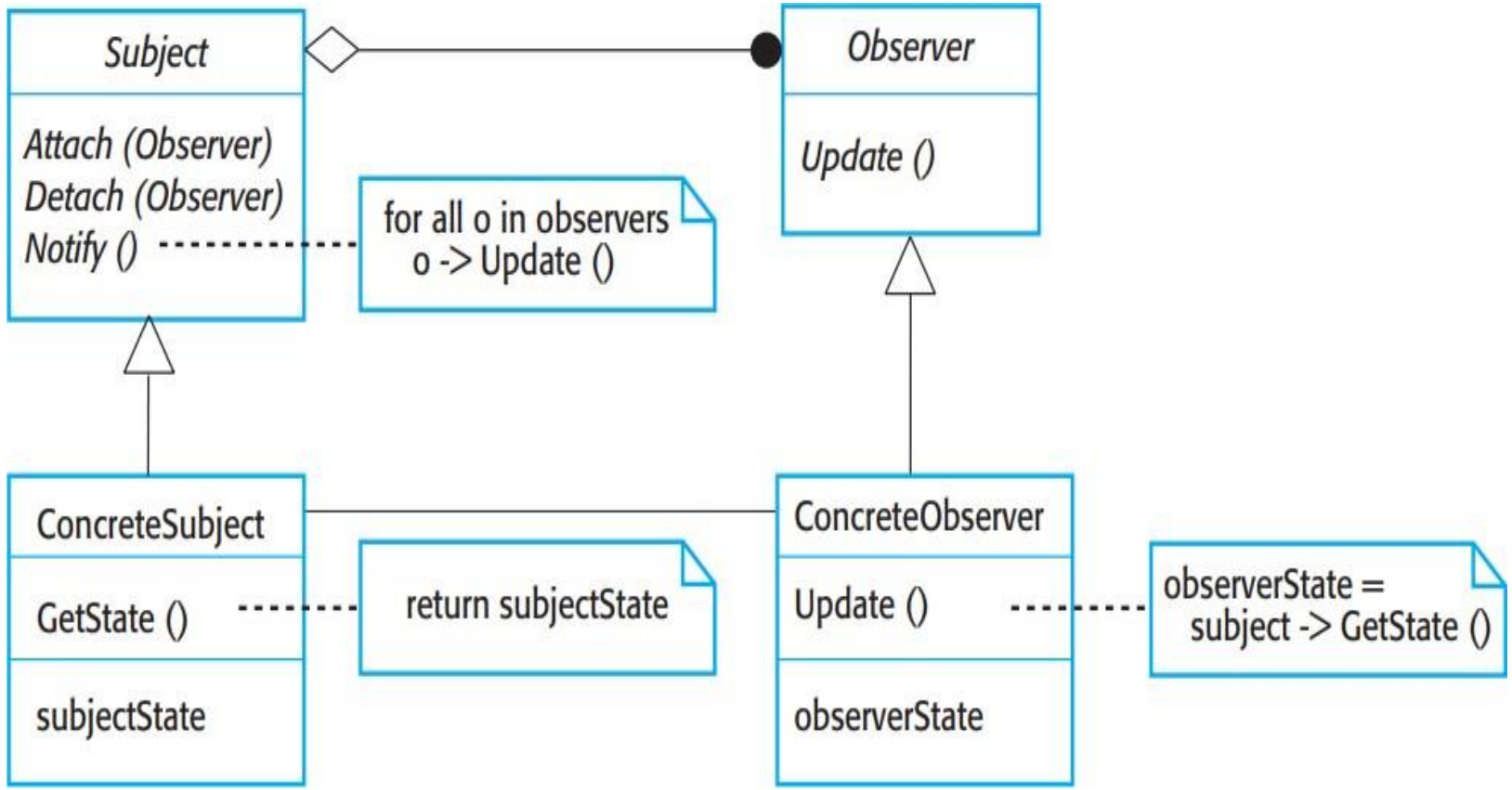
3. Problem Description
   - In many situations, you have to provide multiple displays of state information, such as a graphical display and a tabular display. Not all of these may be known when the information is specified. All alternative presentations should support interaction and, when the state is changed, all displays must be updated.
   - This pattern may be used in all situations where more than one display format for state information is required and where it is not necessary for the object that maintains the state information to know about the specific display formats used.

Design Pattern: Example

4. Solution Description
   - This involves **two abstract objects**, **Subject and Observer**, and two concrete objects, **ConcreteSubject** and **ConcreteObject**, which inherit the attributes of the related abstract objects. The abstract objects include general operations that are applicable in all situations. The state to be displayed is maintained in ConcreteSubject, which inherits operations from Subject allowing it to add and remove Observers (each observer corresponds to a display) and to issue a notification when the state has changed.

Design Pattern: Example

- A UML model of the Observer pattern



Design Pattern: Example

5. Consequences
   - The subject only knows the **abstract Observer** and does not know details of the concrete class. Therefore there is minimal coupling between these objects. Because of this lack of knowledge, optimizations that enhance display performance are impractical. Changes to the subject may cause a set of linked updates to observers to be generated, some of which may not be necessary.

Design Pattern: Example

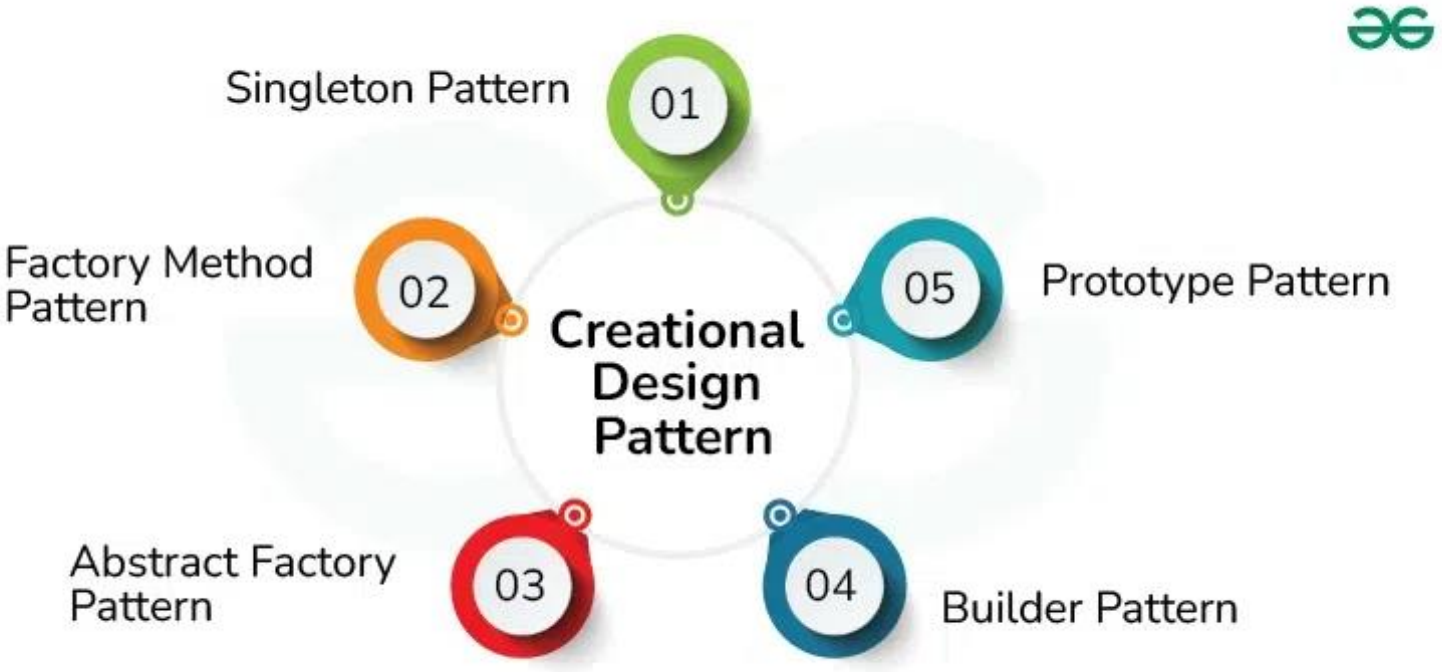
- The Observer pattern
  - Name: Observer.
  - Description: Separates the display of object state from the object itself.
  - Problem description: Used when multiple displays of state are needed.
  - Solution description: See slide with UML description.
  - Consequences: Optimizations to enhance display performance are impractical.

Design Pattern Types

- Creational Design Patterns
- Structural Design Patterns
- Behavioral Design Patterns

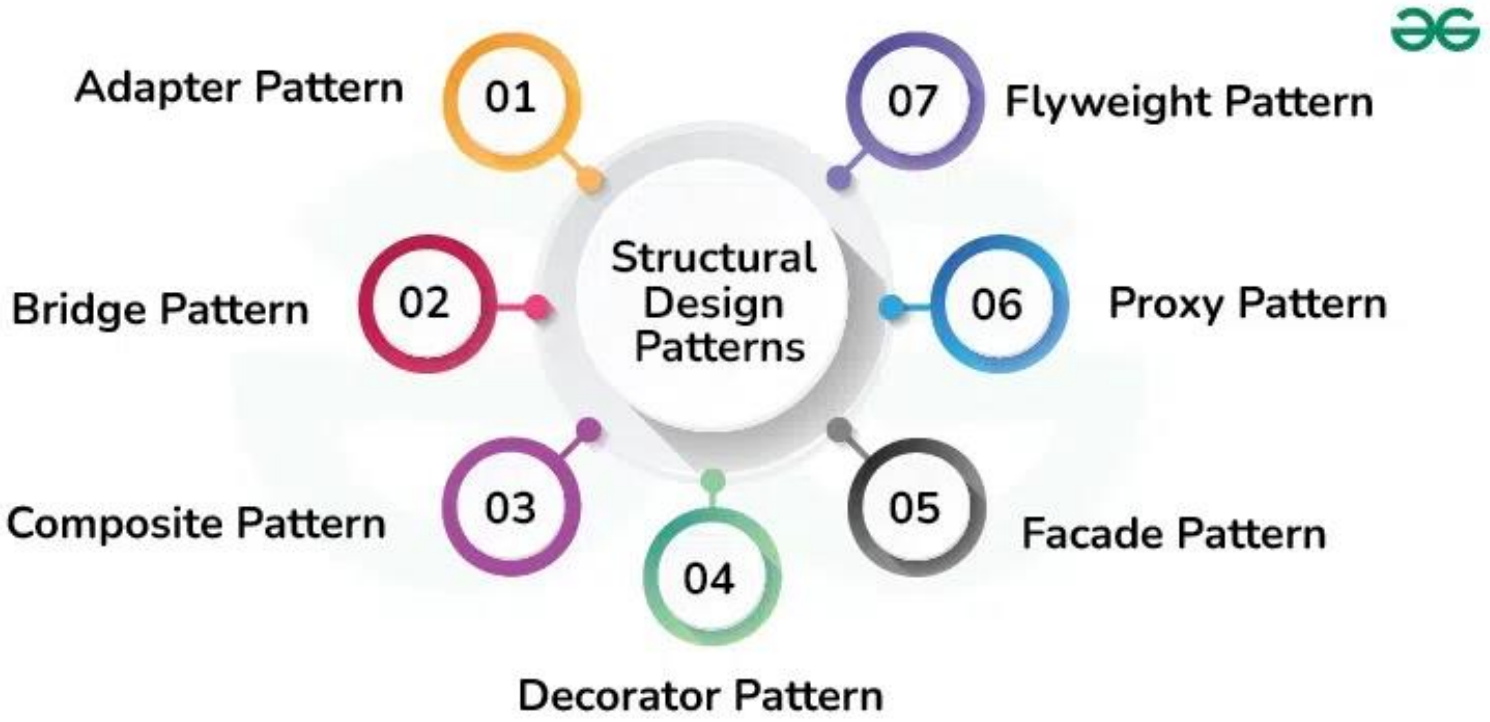
Creational Design Patterns

- *Creational Design Pattern abstract the instantiation process. They help in making a system independent of how its objects are created, composed and represented.*



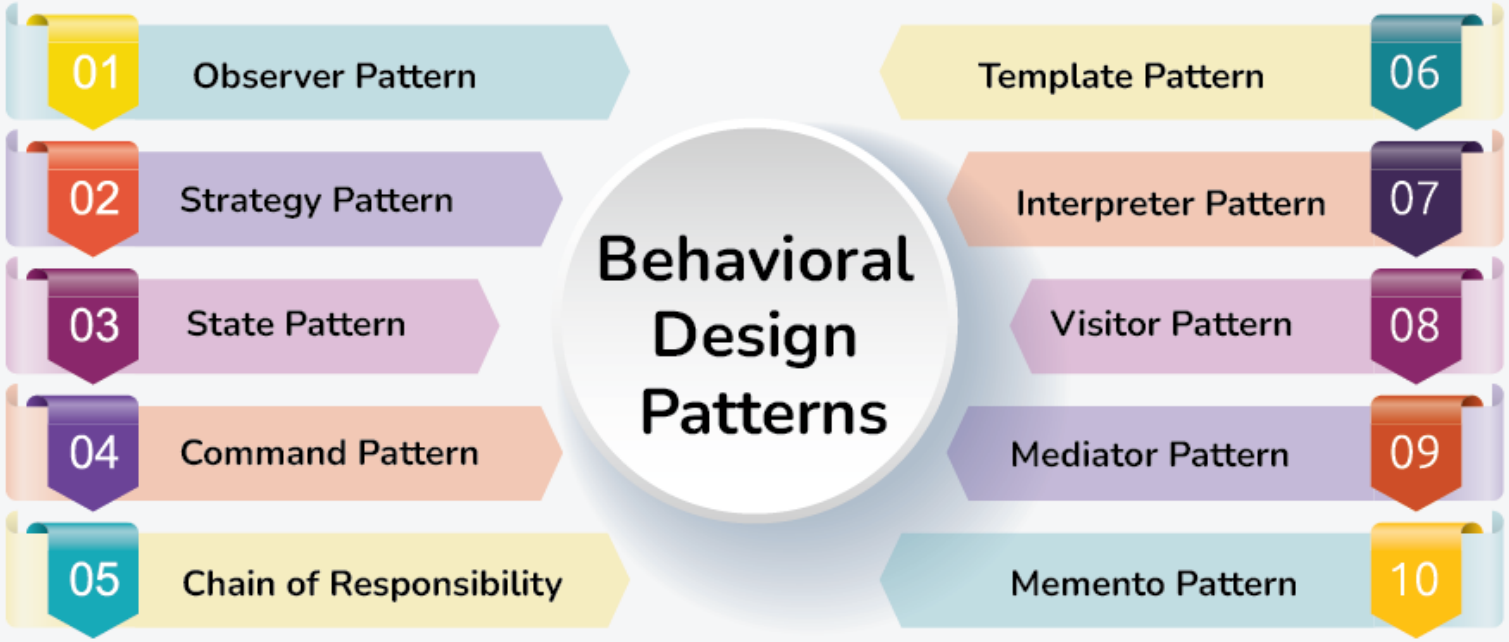
Structural Design Patterns

- *Structural Design Patterns are concerned with how classes and objects are composed to form larger structures. Structural class patterns use inheritance to compose interfaces or implementations.*



Behavioral Design Patterns

- *Behavioral Patterns are concerned with algorithms and the assignment of responsibilities between objects. Behavioral patterns describe not just patterns of objects or classes but also the patterns of communication between them. These patterns characterize complex control flow that's difficult to follow at run-time.*



Design problems

- To use patterns in your design, you need to recognize the design problem you are facing
  - Tell several objects that the state of some other object has changed (Observer pattern).
  - Tidy up the interfaces to a number of related objects that have often been developed incrementally (Façade pattern).
  - Provide a standard way of accessing the elements in a collection, irrespective of how that collection is implemented (Iterator pattern).
  - Allow for the possibility of extending the functionality of an existing class at run-time (Decorator pattern).

Implementation Issues

What – Implementation Issues?

- Reuse
- Configuration Management
- Host-Target Management

Implementation Issues

- Focus here is not on programming, although this is obviously important, but on other implementation issues that are often not covered in programming texts:
  - Reuse Most modern software is constructed by reusing existing components or systems. When you are developing software, you should make as much use as possible of existing code.
  - Configuration management During the development process, you have to keep track of the many different versions of each software component in a configuration management system.
  - Host-target development Production software does not usually execute on the same computer as the software development environment. Rather, you develop it on one computer (the host system) and execute it on a separate computer (the target system).

Implementation Issues

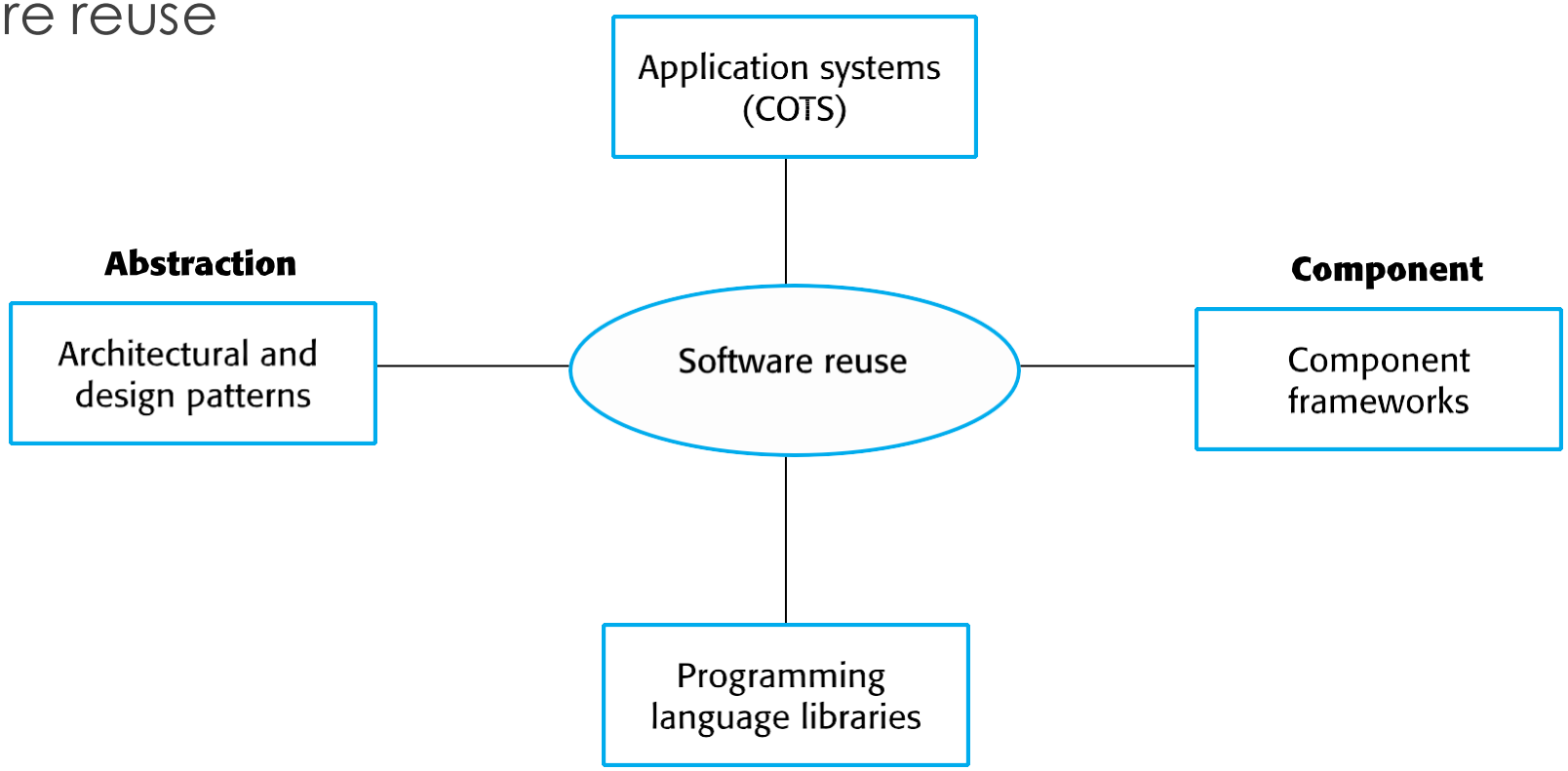
- Reuse
  - From 1960s to 1990s, most new software was developed from scratch, by writing all code in a high- level programming language.
    - The only significant reuse or software was the reuse of functions and objects in programming language libraries.
  - Costs and schedule pressure mean that this approach became increasingly unviable, especially for commercial and Internet-based systems.
  - An approach to development based around the reuse of existing software emerged and is now generally used for business and scientific software.

Implementation Issues

- Reuse Levels
  - The abstraction level
    - At this level, you don't reuse software directly but use knowledge of successful abstractions in the design of your software.
  - The object level
    - At this level, you directly reuse objects from a library rather than writing the code yourself.
  - The component level
    - Components are collections of objects and object classes that you reuse in application systems.
  - The system level
    - At this level, you reuse entire application systems.

Implementation Issues

- Software reuse



Implementation Issues

- Reuse Costs
  - The **costs of the time spent** in looking for software to reuse and assessing whether or not it meets your needs.
  - Where applicable, the **costs of buying** the reusable software. These costs can be very high.
  - The **costs of adapting and configuring** the reusable software components or systems to reflect the requirements of the system that you are developing.
  - The **costs of integrating** reusable software elements with each other (if you are using software from different sources) and with the new code that you have developed.

Implementation Issues

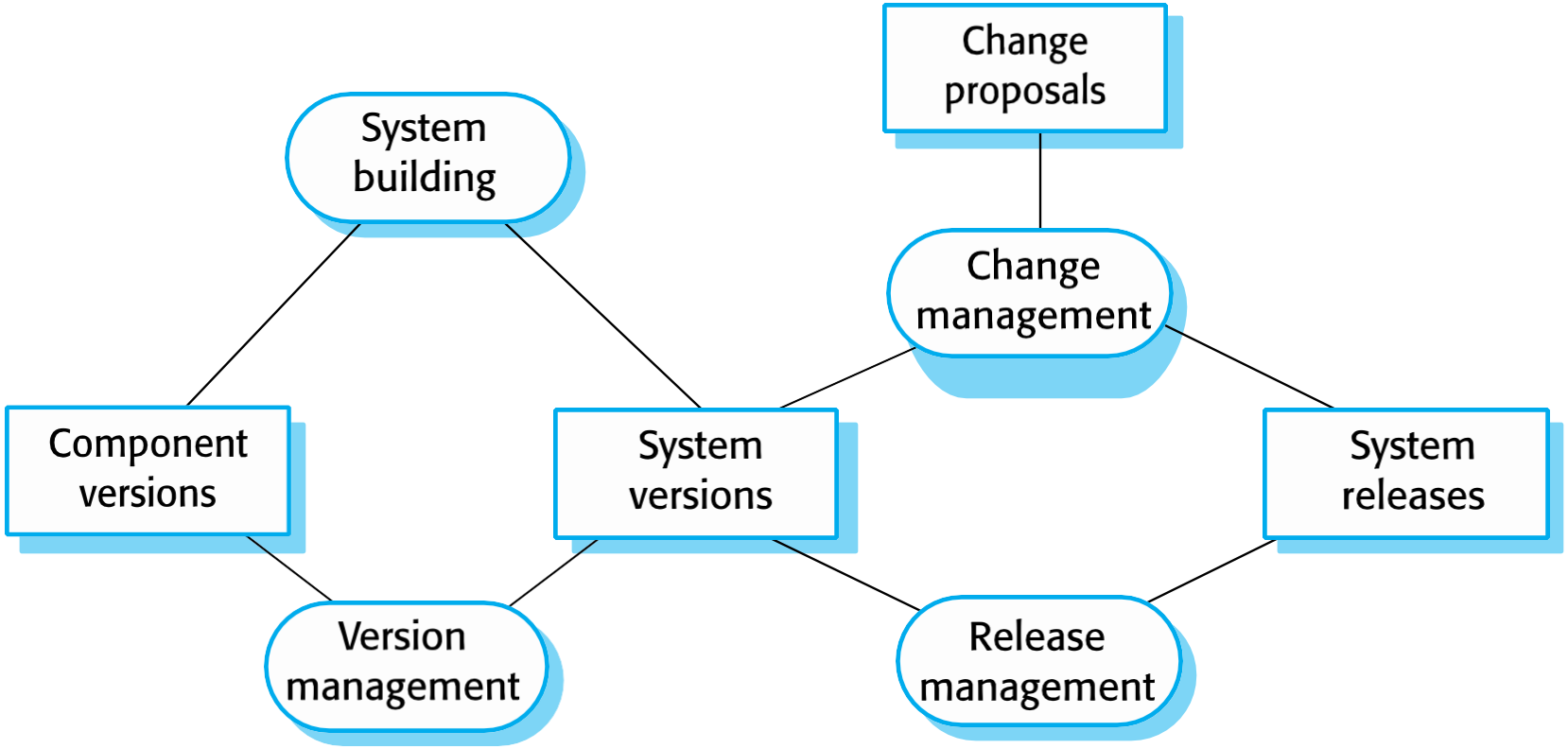
- Configuration Management
  - The process of **managing a changing** software system.
  - The aim of configuration management is to **support the system integration process** so that all developers can access the project code and documents in a controlled way, find out what changes have been made, and compile and link components to create a system.

Implementation Issues

- Configuration management activities
  - **Version management**, where support is provided to keep track of the different versions of software components. Version management systems include facilities to coordinate development by several programmers.
  - **System integration**, where support is provided to help developers define what versions of components are used to create each version of a system. This description is then used to build a system automatically by compiling and linking the required components.
  - **Problem tracking**, where support is provided to allow users to report bugs and other problems, and to allow all developers to see who is working on these problems and when they are fixed.

Implementation Issues

- Configuration management tool interaction

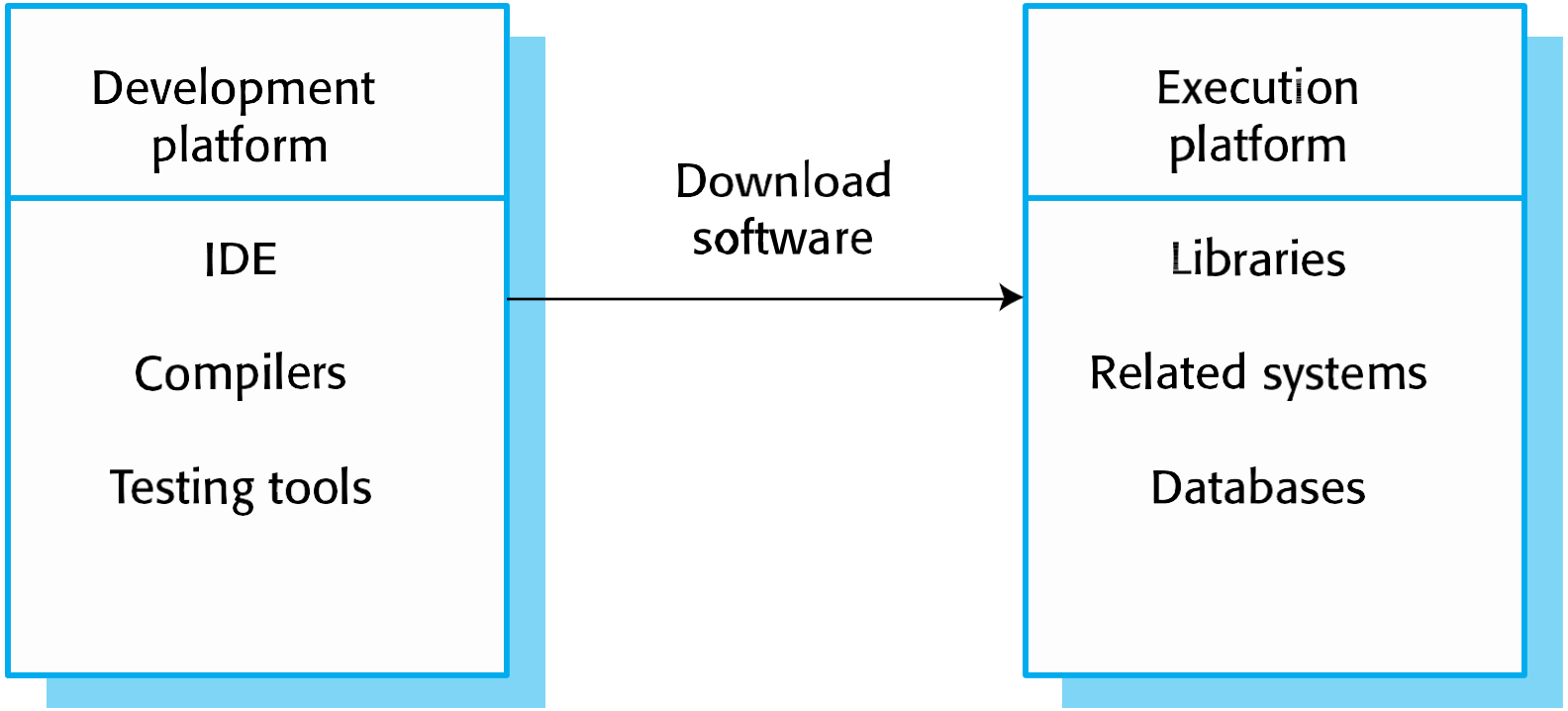


Implementation Issues

- Host-target development
  - Most software is **developed** on one computer (the **host**), but **runs** on a separate machine (the **target**).
  - More generally, we can talk about a development platform and an execution platform.
    - A platform is more than just hardware.
    - It includes the installed operating system plus other supporting software such as a database management system or, for development platforms, an interactive development environment.
  - Development platform usually has different installed software than execution platform; these platforms may have different architectures.

Implementation Issues

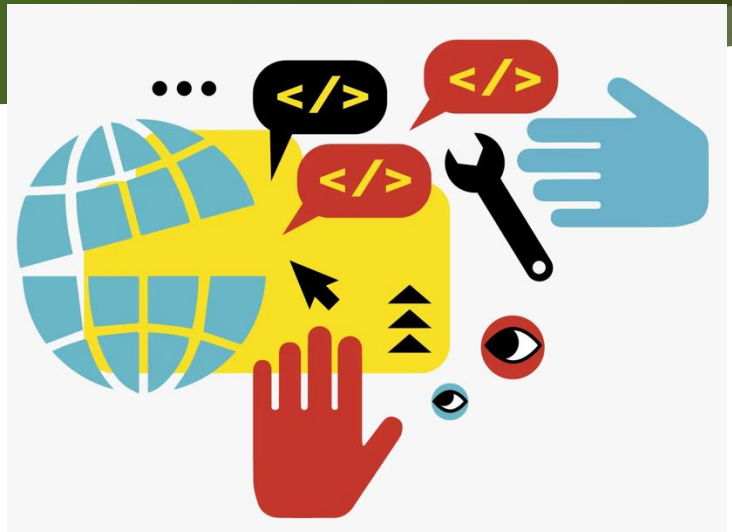
- Host-target development



Implementation Issues

- Development platform tools
  - An integrated compiler and syntax-directed editing system that allows you to create, edit and compile code.
  - A language debugging system.
  - Graphical editing tools, such as tools to edit UML models.
  - Testing tools, such as Junit that can automatically run a set of tests on a new version of a program.
  - Project support tools that help you organize the code for different development projects.

Open-Source Development



Open Source Development

- Open-source development is an **approach to software development** in which the **source code of a software system is published, and volunteers are invited to participate in the development process**
- Its roots are in the **Free Software Foundation** (www.fsf.org), which advocates that source code should not be proprietary but rather should always be available for users to examine and modify as they wish.
- Open-source software extended this idea by using the Internet to recruit a much larger population of volunteer developers. Many of them are also users of the code.

Open-Source Systems

- The best-known open-source product is, of course, the Linux operating system which is widely used as a server system and, increasingly, as a desktop environment.
- Other important open-source products are Java, the Apache web server and the mySQL database management system.

Open-Source Business

- More and more product companies are using an open-source approach to development.
- Their business model is not reliant on selling a software product but on selling support for that product.
- They believe that involving the open-source community will allow software to be developed more cheaply, more quickly and will create a community of users for the software.

Open-Source Licensing

- A fundamental principle of open-source development is that source code should be freely available, this does not mean that anyone can do as they wish with that code.
  - Legally, the **developer of the code** (either a company or an individual) **still owns the code**. They can place restrictions on how it is used by including legally binding conditions in an open-source software license.
  - Some open-source developers believe that **if an open-source component is used** to develop a new system, then that system should also be **open source**.
  - Others are willing to allow their code to be used without this restriction. The developed systems may be proprietary and sold as closed source systems.

Different types of Open-Source License

- **GPL** - The GNU General Public License
- **LGPL** - The GNU Lesser General Public License
- **BSD** - The Berkley Standard Distribution License

Open-Source License Models

- **The GNU General Public License (GPL).** This is a so-called 'reciprocal' license that means that if ***you use open-source software that is licensed under the GPL license, then you must make that software open source.***
- **The GNU Lesser General Public License (LGPL)** is a variant of the GPL license where you ***can write components that link to open-source code without having to publish the source of these components.***
- **The Berkley Standard Distribution (BSD) License**. This is a non-reciprocal license, which means ***you are not obliged to re-publish any changes or modifications made to open-source code.*** You can include the code in proprietary systems that are sold.

Open-Source Issues

- Should the product that is being developed make use of open-source components?
- Should an open-source approach be used for the software's development?

License Management

- Establish a system for maintaining information about open-source components that are downloaded and used.
- Be aware of the different types of licenses and understand how a component is licensed before it is used.
- Be aware of evolution pathways for components.
- Educate people about open source.
- Have auditing systems in place.
- Participate in the open-source community.

Summary

- Software design and implementation are interleaved activities.
- The process of object-oriented design includes the activities to
  - design the system architecture, identify objects in the system, describe the design using different object models, and document the component interfaces.
- A range of different models may be produced during an object-oriented design process. These include
  - static models (class models, generalization models, association models) and dynamic models (sequence models, state machine models).

Summary

- When developing software, you should always consider the possibility of reusing existing software, either as components, services, or complete systems.
- Configuration management is the process of managing changes to an evolving software system.
  - It is essential when a team of people is cooperating to develop software.
- Open-source development involves making the source code of a system publicly available.
  - Many people can propose changes and improvements to the software.